

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 18 settembre 2012 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. È data l'espressione regolare R seguente, di alfabeto $\{a, b\}$:

$$R = \left(a a \mid a b \mid b (b b)^* \right)^+$$

Si risponda alle domande seguenti:

- (a) Si elenchino tutte le stringhe di lunghezza minore o uguale a tre generate dall'espressione regolare R , in ordine alfabetico con $a < b$. Si dia una descrizione informale (a parole) sintetica del linguaggio regolare $L(R)$.
 - (b) Tramite un metodo a scelta (sistematico o intuitivo) si trovi un automa A deterministico che riconosce il linguaggio regolare $L(R)$.
 - (c) (facoltativa) Si stabilisca se l'espressione regolare R è ambigua o no, motivando la risposta, e nel caso lo sia si trovi un'espressione regolare R' non ambigua equivalente a R .
-

Soluzione

- (a) Ecco le stringhe di lunghezza minore o uguale a tre, che in totale sono nove:

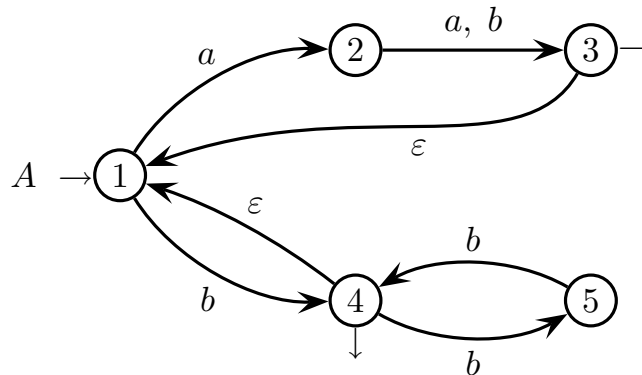
b	$a a$	$a b$	$b b$	
$a a b$	$a b b$	$b a a$	$b a b$	$b b b$

Prendendo spunto dagli esempi (cui si può aggiungere $b a a b$) si vede che il linguaggio $L(R)$ non ha ε ed è formato da tutte le stringhe di a e b , con la sola restrizione che la sequenza finale di lettere a - se esiste - abbia lunghezza pari.

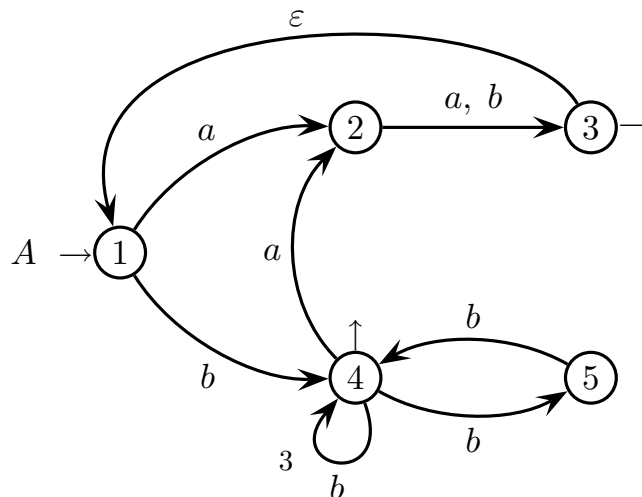
- (b) Si può applicare il metodo di Berri-Sethi e ottenere l'automa deterministico A direttamente dall'espressione regolare R . Qui tuttavia è facile procedere anche per via intuitiva. Intanto, raccogliendo a fattore comune si può riscrivere l'espressione regolare R , così:

$$\left(a (a \mid b) \mid b (b b)^* \right)^+$$

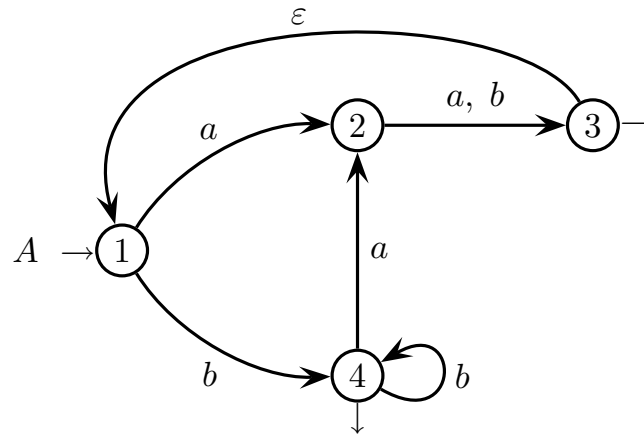
Da questa forma è facile ottenere un automa nondeterministico, che usa ε -transizioni, tramite il metodo di Thompson un po' abbreviato, così:



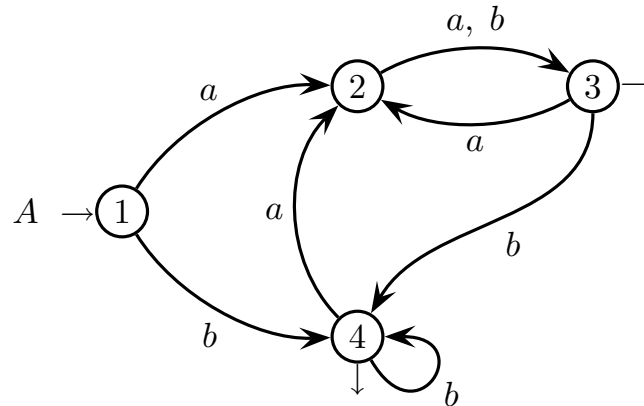
Per rendere deterministico l'automa A , si tagliano le ε -transizioni. Prima si taglia la transizione $4 \xrightarrow{\varepsilon} 1$, duplicando sul nodo 4 gli archi uscenti dal nodo 1:



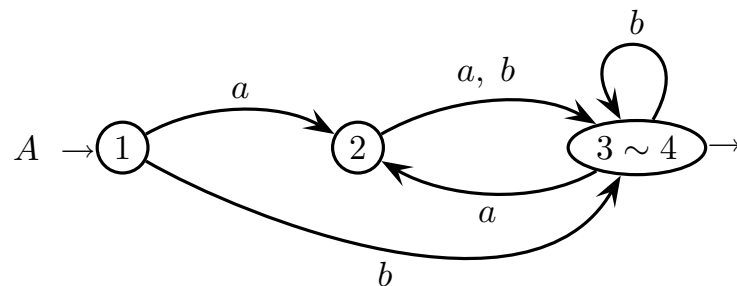
Però il nodo 4 ha archi uscenti duplicati. Tuttavia partendo da 4, con l'autoanello b si riconosce b^* e con il circuito passante per il nodo 5 si riconosce $(bb)^*$, che è contenuto in b^* . Dunque il nodo 5 è ridondante e si può eliminare con tutti gli archi appoggiati, così:



Il nodo 4 non ha più archi uscenti duplicati. Poi si taglia la transizione $3 \xrightarrow{\varepsilon} 1$, duplicando sul nodo 3 gli archi uscenti dal nodo 1:



Questo automa A è deterministico. Si vede subito che gli stati finali 3 e 4 sono indistinguibili e che dunque si possono unire minimizzando l'automa, così:



Questo automa A è minimo poiché dagli stati 1 e 2 con a si va in uno stato non finale e in uno finale, rispettivamente, e dunque gli stati 1 e 2 sono distinguibili.

(c) L'espressione regolare R è ambigua. Se ne numerino i generatori così:

$$R_{\#} = \left(a_1 a_2 \mid a_3 b_4 \mid b_5 (b_6 b_7)^* \right)^+$$

Risolvendo diversamente gli operatori croce e stella, e scegliendo sempre il componente $b_5 (b_6 b_7)^*$, si deriva la stringa $b^3 = b b b$ (che è la stringa ambigua più corta) in due modi differenti. Eccoli:

$$\left(b_5 (b_6 b_7)^0 \right)^3 = b_5 b_5 b_5$$

e

$$\left(b_5 (b_6 b_7)^1 \right)^1 = b_5 b_6 b_7$$

Gli altri due componenti di R e le loro combinazioni, palesemente non danno luogo ad ambiguità poiché le condizioni per avere ambiguità di concatenamento non sono soddisfatte.

Resta dunque da trovare un'espressione regolare R' non ambigua equivalente a R . Naturalmente si potrebbe ottenere l'espressione R' dall'automa deterministico A trovato prima, per esempio tramite eliminazione dei nodi. Ma qui è facile anche procedere intuitivamente. Basta osservare che la sotto-espressione ambigua $\left(b (b b)^* \right)^+$ genera il linguaggio b^+ . Dunque l'espressione R si semplifica così:

$$R' = \left(a a \mid a b \mid b \right)^+$$

L'espressione R' è equivalente a R e palesemente non è ambigua, avendo disambiguata l'unica parte ambigua di R .

Forme derivate da questa, anch'esse non ambigue, sono le seguenti:

$$R' = \left(a (a \mid b) \mid b \right)^+$$

oppure

$$R' = \left(a a \mid [a] b \right)^+$$

dove le parentesi quadre indicano opzionalità.

2 Grammatiche libere e automi a pila 20%

1. Si risponda alle domande seguenti, argomentando tutte le risposte:

- (a) Si consideri un linguaggio di alfabeto $\{a, b, c\}$ formato da stringhe di lunghezza dispari e con una c al centro, tali che se in una posizione prima della c si trova una a (rispettivamente, una b) allora dopo la c , in posizione simmetrica, si trova una b (rispettivamente, una a), e viceversa. Esempi di stringhe valide sono:

$abbc aab \qquad babbcaaba$

Si scriva una grammatica per tale linguaggio e si stabilisca se esso è riconoscibile da un automa a pila deterministico.

- (b) Si consideri il linguaggio di alfabeto $\{a, b\}$ costituito da stringhe di lunghezza multipla di 4 che sono sequenze di coppie di tipo aa e bb , tali che se in una posizione prima del centro della stringa si trova una a (rispettivamente, una b) allora dopo il centro, in posizione simmetrica, si trova una b (rispettivamente, una a), e viceversa. Esempi di stringhe valide sono:

$bb a a a b b b b a a \qquad a a b b a a a b b b b a a b b$

Si scriva una grammatica per tale linguaggio e si stabilisca se esso è riconoscibile da un automa a pila deterministico.

- (c) (facoltativa) Si consideri la grammatica G non estesa seguente (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow X \mid Y \\ X \rightarrow a Y b \mid a b \\ Y \rightarrow b X a \mid b a \end{array} \right.$$

Si stabilisca qual è l'automa deterministico più semplice che riconosce il linguaggio generato dalla grammatica G .

Soluzione

- (a) Il linguaggio è generato dalla grammatica seguente (assioma S):

$$\begin{cases} S \rightarrow a S b \\ S \rightarrow b S a \\ S \rightarrow c \end{cases}$$

ed è riconoscibile da un automa a pila deterministico molto semplice, che impila la parte di stringa a sinistra di c e che leggendo la parte a destra di c verifica se i caratteri in posizione simmetrica rispetto a c corrispondono.

- (b) Il linguaggio è generato dalla grammatica seguente (assioma S):

$$\begin{cases} S \rightarrow a a S b b \\ S \rightarrow b b S a a \\ S \rightarrow \varepsilon \end{cases}$$

ed è riconoscibile da un automa a pila nondeterministico che opera come prima, seppure decidendo nondeterministicamente dove si trova il centro della stringa (non segnalato da alcun marcatore); ma non è riconoscibile da uno deterministico che non saprebbe come identificare la posizione centrale nella stringa.

- (c) Il linguaggio $L(G)$ è composto da stringhe di lunghezza pari (maggiore o uguale a due) con simboli a e b alternati, e palesemente è regolare, dunque riconoscibile da un automa a stati finiti deterministico.

Per convincersi basta seguire qualche derivazione. Per esempio considerando il nonterminale X si ha:

$$X \Rightarrow a b$$

$$X \Rightarrow a Y b \Rightarrow a b a b$$

$$X \Rightarrow a Y b \Rightarrow a b X a b \Rightarrow a b a b a b$$

...

e similmente per Y scambiando a e b . Dunque ne viene $L(X) = (a b)^+$ e $L(Y) = (b a)^+$, pertanto si ha $L(G) = L(S) = (a b)^+ \mid (b a)^+$ e l'automa finito deterministico riconosce quest'espressione regolare.

2. Si considerino gli esempi seguenti di espressione booleana ispirati allo stile dello storico linguaggio di programmazione Algol (a cavallo tra gli anni '50 e '60):

$x > z$	operatore relazionale
$y > x \vee z < w$	
$p \wedge q \vee x > y$	
$p \wedge (q \vee x > y)$	si possono usare le parentesi
$g \supset \neg q \wedge b \wedge \neg c \vee d \vee e \supset \neg f$	
if $x < w$ then $y > w$ else $p \vee q$	in if il ramo else è obbligatorio
if if q then b else f then g else $x < z$	gli if possono essere annidati

I simboli x, y, w, z designano variabili aritmetiche; gli altri simboli (p, q, g, b, c , etc.) designano variabili booleane. Gli operatori seguenti sono ammessi nelle espressioni:

operatori relazionali: $> <$ si applicano a variabili aritmetiche

operatori logici: $\neg \wedge \vee \supset$ (implicazione) si applicano a variabili ed espressioni booleane

precedenze tra operatori: l'ordine di applicazione degli operatori di un'espressione è *generalmente da sinistra a destra*, con le seguenti regole aggiuntive di precedenza:

- first (cioè massima priorità): $< >$
- second: $\neg$
- third: $\wedge$
- fourth: $\vee$
- fifth: $\supset$
- sixth: **if** ... **then** ... **else** ...

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G , in forma BNF (non estesa) e non ambigua, per definire il linguaggio delle espressioni booleane tratteggiato sopra. Si indichino le variabili aritmetiche con il carattere *terminale* **a** e le variabili booleane con il carattere *terminale* **b**.
- (b) Si disegni l'albero sintattico dell'espressione booleana seguente:

if $(x < w \supset p) \vee q$ **then** q **else** $p \wedge q$

Soluzione

- (a) Ecco la sintassi ufficiale di Algol 60 per la parte rilevante (assioma $\langle \text{boolean_expr} \rangle$). Si noti che qui compare anche l'operatore di equivalenza logica $\equiv$, la cui precedenza sta tra implicazione e condizionale.

$\langle \text{boolean_expr} \rangle$	$\rightarrow$	$\langle \text{if_clause} \rangle \langle \text{simple_boolean} \rangle \text{ else } \langle \text{boolean_expr} \rangle$ $ \langle \text{simple_boolean} \rangle$
$\langle \text{if_clause} \rangle$	$\rightarrow$	if $\langle \text{boolean_expr} \rangle$ then
$\langle \text{simple_boolean} \rangle$	$\rightarrow$	$\langle \text{simple_boolean} \rangle \equiv \langle \text{implication} \rangle$ $ \langle \text{implication} \rangle$
$\langle \text{implication} \rangle$	$\rightarrow$	$\langle \text{implication} \rangle \supset \langle \text{boolean_term} \rangle$ $ \langle \text{boolean_term} \rangle$
$\langle \text{boolean_term} \rangle$	$\rightarrow$	$\langle \text{boolean_term} \rangle \vee \langle \text{boolean_factor} \rangle$ $ \langle \text{boolean_factor} \rangle$
$\langle \text{boolean_factor} \rangle$	$\rightarrow$	$\langle \text{boolean_factor} \rangle \wedge \langle \text{boolean_secondary} \rangle$ $ \langle \text{boolean_secondary} \rangle$
$\langle \text{boolean_secondary} \rangle$	$\rightarrow$	$\langle \text{boolean_primary} \rangle \neg \langle \text{boolean_primary} \rangle$
$\langle \text{boolean_primary} \rangle$	$\rightarrow$	$(\langle \text{boolean_expr} \rangle) \langle \text{relation} \rangle \mathbf{b}$
$\langle \text{relation} \rangle$	$\rightarrow$	$\mathbf{a} \langle \text{relational_operator} \rangle \mathbf{a}$
$\langle \text{relational_operator} \rangle$	$\rightarrow$	$< >$

La grammatica ufficiale di Algol impone alle espressioni booleane che figurano come argomento del ramo **then** o **else** del condizionale **if**, d'essere racchiuse tra parentesi così da attribuirle non ambigualmente al ramo dove figurano; a meno che l'espressione nel ramo non sia uno **if** o una relazione aritmetica o una variabile logica, poiché in tale caso l'attribuzione è chiara anche senza parentesi. Pertanto la grammatica ufficiale di Algol non genera l'esempio dato nel testo così com'è, ma richiede di parentetizzarlo e interpretarlo così:

$$\underbrace{\mathbf{if} \left(x < w \supset p \right) \vee q \mathbf{then} q \mathbf{else} \left(p \wedge q \right)}_{\text{oppure}}$$

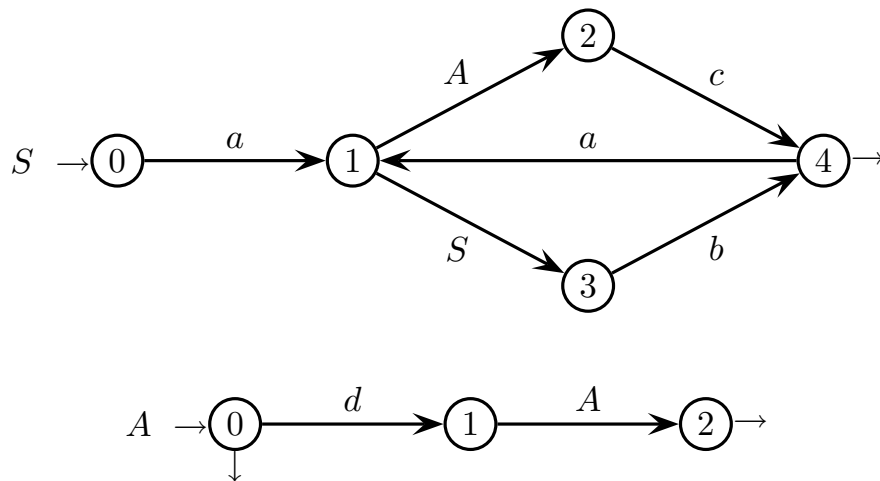
$$\underbrace{\left(\mathbf{if} \left(x < w \supset p \right) \vee q \mathbf{then} q \mathbf{else} p \right) \wedge q}_{\text{oppure}}$$

Naturalmente ci possono essere anche altre soluzioni che trattano i rami **then** e **else** diversamente rispetto alla specifica ufficiale di Algol, poiché gli esempi nel testo lasciano il lettore abbastanza libero di completare la specifica.

- (b) L'albero sintattico come da grammatica ufficiale di Algol e secondo una delle due interpretazioni possibili, è facile ed è lasciato al lettore.

3 Analisi sintattica e parsificatori 20%

1. È data la grammatica estesa G seguente, espressa come rete seguente di macchine ricorsive (assioma S) di alfabeto terminale $\{a, b, c, d\}$ e alfabeto nonterminale $\{S, A\}$:



Si risponda alle domande seguenti (mediante la metodologia di analisi estesa):

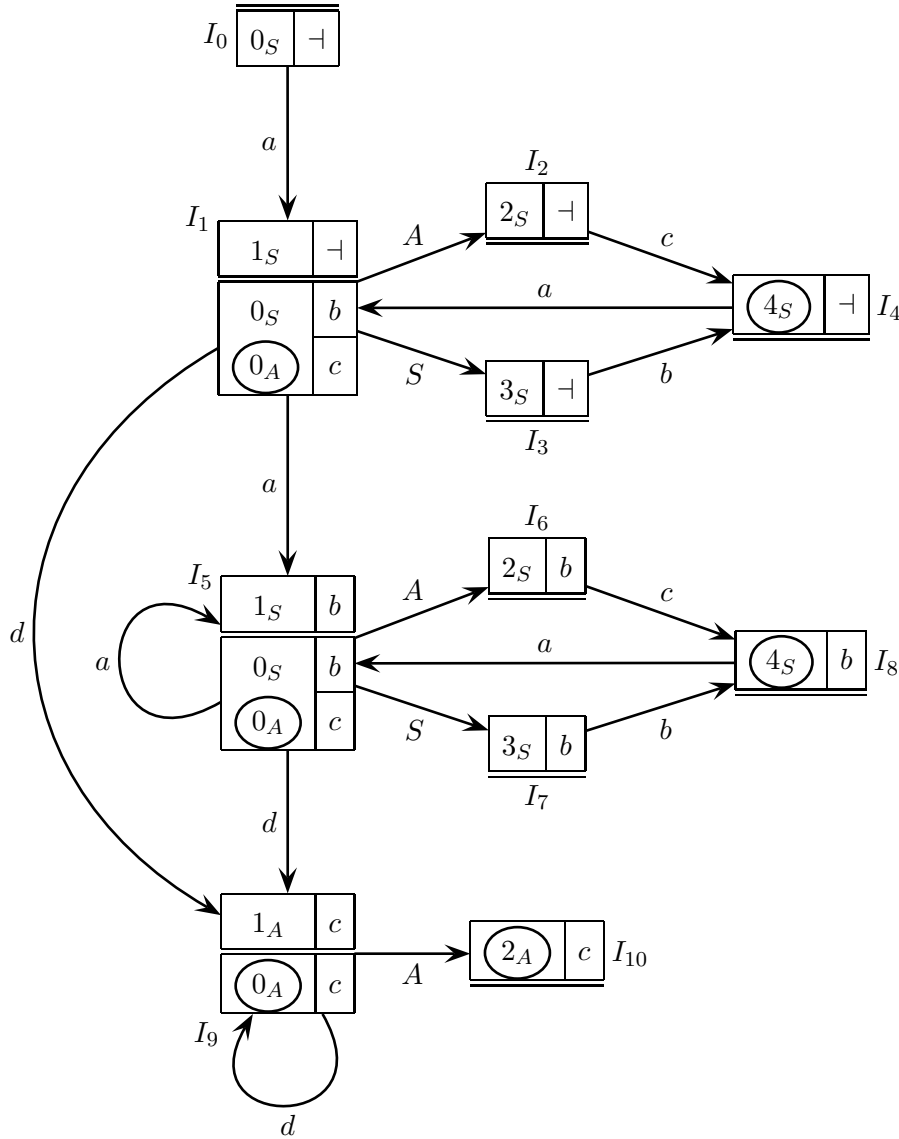
- (a) Si dica se la grammatica G è di tipo $ELR(1)$, motivando la risposta.
- (b) Si dica se la grammatica G è di tipo $ELL(1)$, motivando la risposta.
- (c) (facoltativa) Si costruisca la procedura S dell'analizzatore sintattico ELL di G , scrivendone lo pseudo-codice.

Nota: chi preferisse rispondere mediante la metodologia di analisi classica LR , utilizzi la grammatica non estesa seguente, equivalente a G (assioma S_0):

$$\left\{ \begin{array}{l} S_0 \rightarrow S \mid S S_0 \\ S \rightarrow a A c \mid a S b \\ A \rightarrow \varepsilon \mid d A \end{array} \right.$$

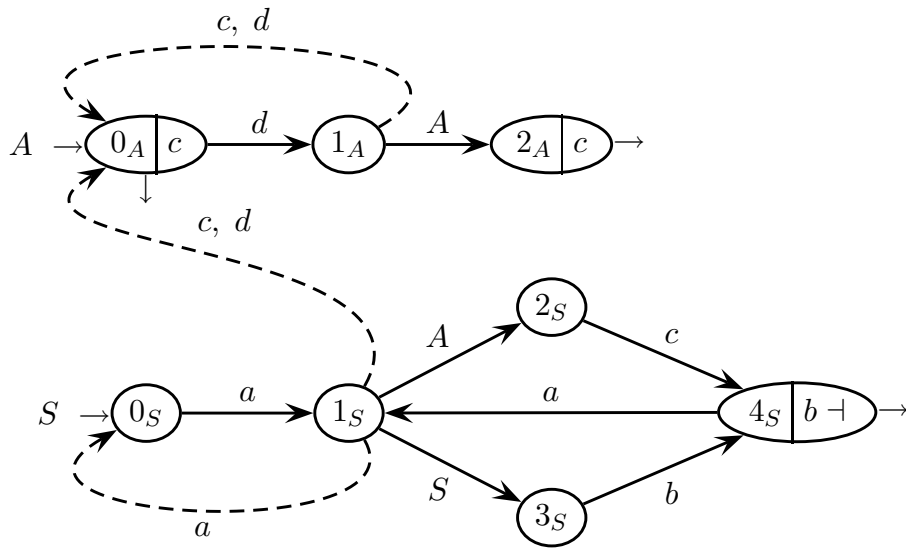
Soluzione

(a) Ecco il grafo pilota *ELR* della grammatica *G*.



Poiché non ci sono conflitti *ELR* (spostamento-riduzione o riduzione-riduzione), la grammatica *G* soddisfa la condizione *ELR* e pertanto è di tipo *ELR*(1).

- (b) Il grafo pilota di *G* soddisfa la condizione *ELR*, soddisfa la condizione di Beatty e la grammatica *G* non ha ricorsioni sinistre; dunque la grammatica *G* soddisfa la condizione *ELL* e pertanto è di tipo *ELL*(1).
- (c) Ecco il grafo pilota semplificato *ELL* (tracciato direttamente), con gli archi di chiamata, i guide set e i look-ahead set.



Ben si vede che il pilota *ELL* è deterministico, come si sapeva già. Ecco lo pseudo-codice della procedura a discesa ricorsiva associata alla macchina *S*.

```

procedure S
    if c = a then                                – state 0
        next                                         – goto 1
    LOOP:      if c ∈ { c, d } then                – state 1
        call A                                       – goto 2
        if c = c then                                – state 2
            next                                         – goto 4
            if c ∈ { b, ⊥ } then return                – state 4
            else if c = a then                        – state 4
                next                                         – goto 1
                goto LOOP                                – state 1
            else error                                    – state 4
        else error                                       – state 2
    else if c = a then                                – state 1
        call S                                       – goto 2
        if c = b then                                – state 2
            next                                         – goto 4
            if c ∈ { b, ⊥ } then return                – state 4
            else if c = a then                        – state 4
                next                                         – goto 1
                goto LOOP                                – state 1
            else error                                    – state 4
        else error                                       – state 2
    else error                                         – state 1
end procedure                                         – state 0

```

Naturalmente si può ottimizzare lo pseudo-codice in vari modi, come accorpare i rami d'errore, rendere il goto con un ciclo ben strutturato e simili altre trasformazioni di codice (si lascia al lettore).

Quanto alla grammatica non estesa equivalente, essa risulta essere *LR*(1) ed *LL*(1). La procedura *S* dell'analizzatore sintattico *LL* è analoga a quella estesa.

4 Traduzione e analisi semantica 20%

1. Si consideri il linguaggio definito dall'espressione regolare $(a^+c)^+$, le cui stringhe sono formate da sequenze di almeno un fattore di tipo $a^n c$ con $n \geq 1$. Si vuole tradurre tali stringhe trasformando ogni fattore $a^n c$ in uno di tipo $b^k s$ con $k \geq 0$, dove si ha $k = \frac{n}{2} - 1$ se n è pari oppure $k = \lceil \frac{n}{2} \rceil$ se n è dispari (per n dispari la notazione $\lceil \frac{n}{2} \rceil$ indica il minimo intero maggiore di $\frac{n}{2}$). Per esempio:

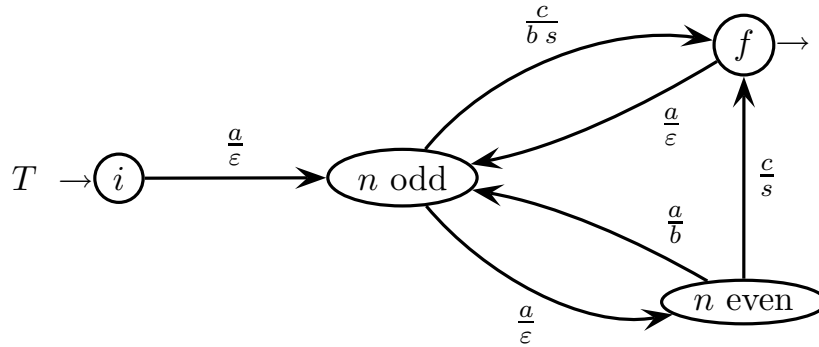
$$\tau(a^3 c a^4 c a^5 c a^2 c) = b b s b s b b b s s$$

Si risponda alle domande seguenti:

- (a) Si dica se la traduzione τ è biunivoca, argomentando la risposta.
 - (b) Si dica se la traduzione τ è calcolabile con un traduttore finito deterministico, argomentando la risposta.
 - (c) Si definisca un automa traduttore finito T che calcoli la traduzione τ , se possibile deterministico.
 - (d) (facoltativa) Si definisca la traduzione τ mediante un'espressione regolare di traduzione R_τ .
-

Soluzione

- (a) La traduzione τ non è biunivoca: un certo numero $k \geq 1$ di lettere destinazione b può risultare da un numero $n = 2(k + 1)$ come pure da un numero $n = 2k - 1$ di lettere sorgente a . Per esempio il fattore destinazione b^2 può derivare da due fattori sorgente distinti: a^6 a a^3 .
- (b) La traduzione τ può essere calcolata da un traduttore finito deterministico. Esso legge il fattore sorgente $a^n c$ formato da lettere a ed emette il numero minimo di lettere destinazione b compatibilmente con ambo i casi pari e dispari; inoltre se la lettera c viene letta quando il numero di a è dispari, emette una b aggiuntiva.
- (c) L'automa traduttore finito T è definito dalla figura seguente, ispirata alla descrizione data nel punto precedente:



L'automa traduttore T è deterministico poiché lo è il riconoscitore soggiacente.

- (d) Ecco l'espressione regolare di traduzione R_τ , derivata direttamente dall'automa traduttore finito T :

$$R_\tau = \left(\frac{a}{\epsilon} \left(\frac{aa}{b} \right)^* \frac{c}{bs} \mid \frac{aa}{\epsilon} \left(\frac{aa}{b} \right)^* \frac{c}{s} \right)^+$$

Un'altra forma, meno direttamente legata all'automa T , è questa:

$$R_\tau = \left(\frac{a}{b} \left(\frac{aa}{b} \right)^* \frac{c}{s} \mid \frac{aa}{\epsilon} \left(\frac{aa}{b} \right)^* \frac{c}{s} \right)^+$$

ossia, raccogliendo a fattore comune:

$$R_\tau = \left(\left(\frac{a}{b} \mid \frac{aa}{\epsilon} \right) \left(\frac{aa}{b} \right)^* \frac{c}{s} \right)^+$$

che è piuttosto breve.

2. Un volo è descritto da una 3-upla della forma seguente:

$$\langle \text{flight} \rangle \rightarrow (\text{flight_number}, \text{destination}, \text{cost})$$

dove *flight_number* è una sigla, *destination* è il nome di un aeroporto e *cost* è un numero (si possono considerare terminali). Un catalogo di voli è descritto da una lista, così:

$$\langle \text{catalogue} \rangle \rightarrow \langle \text{flight} \rangle \langle \text{catalogue} \rangle \mid \varepsilon$$

Per selezionare i voli dal catalogo si scrive un'interrogazione descritta dalla sintassi seguente:

$$\langle \text{query} \rangle \rightarrow \text{select flight_number from } \langle \text{catalogue} \rangle \text{ where } \langle \text{condition} \rangle$$

$$\langle \text{condition} \rangle \rightarrow \text{destination} = \text{airport} \mid \dots$$

Per esempio l'interrogazione seguente:

```
select flight_number from
( LH3, Madrid, 230 ) ( AZ2, Madrid, 305 ) ( AF3, Paris, 250 )
where destination = Madrid
```

dà la risposta:

$$\text{selection} = \{ LH3, AZ2 \}$$

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica con attributi *G* per calcolare la risposta a un'interrogazione. La sintassi è questa (assioma $\langle \text{query} \rangle$):

$$\begin{array}{lcl} \langle \text{query} \rangle & \rightarrow & \text{select flight_number from } \langle \text{catalogue} \rangle \\ & & \text{where } \langle \text{condition} \rangle \\ \hline \langle \text{catalogue} \rangle & \rightarrow & \langle \text{flight} \rangle \langle \text{catalogue} \rangle \\ \hline \langle \text{catalogue} \rangle & \rightarrow & \varepsilon \\ \hline \langle \text{flight} \rangle & \rightarrow & (\text{flight_number}, \text{destination}, \text{cost}) \\ \hline \langle \text{condition} \rangle & \rightarrow & \text{destination} = \text{airport} \end{array}$$

La risposta all'interrogazione va assegnata all'attributo sinistro *selection* dell'assioma $\langle \text{query} \rangle$. Per ciascun attributo scelto per il calcolo, nella tabella della prossima pagina si indichino la categoria dell'attributo (sinistro/destro) e a quali simboli nonterminali o terminali esso appartiene.

Si scrivano le regole semantiche nell'apposto spazio e si disegni l'albero sintattico decorato con i valori degli attributi per l'esempio.

- (b) (facoltativa) Si disegnino i grafi delle dipendenze funzionali per ogni regola e si dica se gli attributi sono calcolabili con il metodo one-sweep (a una scansione).

<i>sx/dx</i>	<i>nome</i>	<i>(non)terminali</i>	<i>tipo</i>	<i>significato</i>
sx	<i>selection</i>	query	insieme di flight_number	risultato della query

Ci sono anche gli attributi (convenzionalmente di tipo destro) che danno i valori dei terminali `flight_number`, `destination`, `cost` e `airport`, e che hanno lo stesso nome dei terminali cui si riferiscono. Sono del tutto ovvi e qui neppure riportati.

$\langle \text{query} \rangle_0 \rightarrow$ select flight_number from $\langle \text{catalogue} \rangle_1$ where $\langle \text{condition} \rangle_2$	
---	--

$\langle \text{catalogue} \rangle_0 \rightarrow$ $\langle \text{flight} \rangle_1 \langle \text{catalogue} \rangle_2$	
--	--

$\langle \text{catalogue} \rangle_0 \rightarrow$ ε	
---	--

$\langle \text{flight} \rangle_0 \rightarrow$ (flight_number ₁ , destination ₂ , cost ₃)	
--	--

$\langle \text{condition} \rangle_0 \rightarrow$ destination = airport ₁	
---	--

Soluzione

- (a) Gli attributi significativi da usare nella grammatica G sono:

<i>cond</i>	left	tipo airport	$\langle \text{condition} \rangle$
<i>dest</i>	right	tipo destination	$\langle \text{catalogue} \rangle \quad \langle \text{query} \rangle$
<i>selection</i>	left	tipo insieme di flight_number	$\langle \text{flight} \rangle \quad \langle \text{catalogue} \rangle \quad \langle \text{query} \rangle$

Oltre a questi ci sono gli attributi ovvi che danno i valori dei terminali *flight_number*, *destination*, *cost* e *airport* (qui l'attributo di *cost* non serve).

Grammatica G con attributi:

$\langle \text{query} \rangle_0$	$\rightarrow$	select flight_number
		from $\langle \text{catalogue} \rangle_1$
		where $\langle \text{condition} \rangle_2$
$selection_0$	$:=$	$selection_1$
$dest_1$	$:=$	$cond_2$
$\langle \text{catalogue} \rangle_0$	$\rightarrow$	$\langle \text{flight} \rangle_1 \quad \langle \text{catalogue} \rangle_2$
$selection_0$	$:=$	$selection_1 \cup selection_2$
$dest_1$	$:=$	$dest_0$
$dest_2$	$:=$	$dest_0$
$\langle \text{catalogue} \rangle_0$	$\rightarrow$	ε
$selection_0$	$:=$	$\emptyset$
$\langle \text{flight} \rangle_0$	$\rightarrow$	(<i>flight_number</i> ₁ , <i>destination</i> ₂ , <i>cost</i> ₃)
$selection_0$	$:=$	if $dest_0 = destination_2$
		then { <i>flight_number</i> ₁ }
		else $\emptyset$
		endif
$\langle \text{condition} \rangle_0$	$\rightarrow$	destination = <i>airport</i> ₁
$cond_0$	$:=$	<i>airport</i> ₁

Volendo si possono lasciare *flight_number*, *destination*, *cost* e *airport* come non-terminali, qui non espansi, che si riferiscono a campi del database di voli, e introdurre una funzione string che ne restituisce il valore estraendolo dal DB.

L'albero sintattico decorato è facile ed è lasciato al lettore.

- (b) I grafi delle dipendenze per ciascuna regola sintattica sono lasciati al lettore.

La grammatica G ha attributi sintetizzati ed ereditati. Comunque gli attributi sono valutabili con una sola scansione. Per convincersi basta esaminare i grafi delle dipendenze funzionali oppure applicare per esteso la condizione one-sweep (a una scansione).